

Same Words, Different Bets

Lexicons, static embeddings and contextual pre-training for sentiment classification under scarce labels and ambiguity



Given 1.2M unlabeled words: ignore them, distil them, or pre-train on them?

SST-2 + IMDb | CPU only | 3 seeds | Anurag Prasad

The problem: seven questions, one protocol

Same data, same head, same evaluation for every model

RQ1-2

Does pre-training help? How robust is it?

RQ3-4

How much pre-training is enough, and what does it cost?

RQ5

How far do public checkpoints move the picture?

RQ6

Given the same text, what's the best family to use it with?

RQ7

Where do errors concentrate under ambiguity, and do models know when they're unsure?

Approach: six families, one protocol

Every model - lexicon to transformer - fine-tuned/fit and evaluated identically

VADER lexicon
0 task labels

TF-IDF word/char
+ LogReg/NB/SVM

word2vec / fastText
same 1.2M words

Tiny-BERT
random init

Tiny-BERT
+MLM pre-train

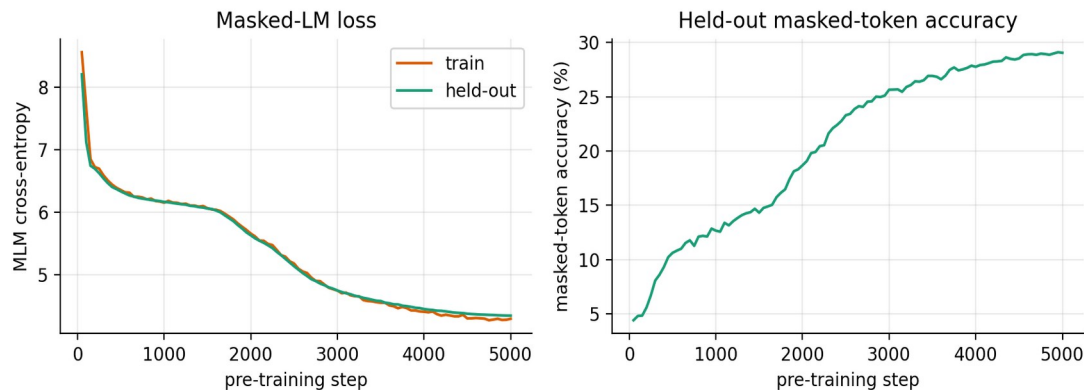
BERT-Tiny /
DistilBERT
public

Evaluated on: clean SST-2, typos (10/30%), word-drop (20/40%), out-of-domain IMDb, and - new - ambiguity strata from fine-grained SST-5 labels (strong vs weak polarity), confidence calibration, and linguistic markers (contrast, negation).

Tiny-BERT: 2 layers, hidden 128, 1,173,248 params. MLM pre-training: 5,000 steps, 13 min, on 1.2M unlabeled IMDb words. All decisions (regularisation, embedding choices) tuned on dev only.

Pre-training starts with a long plateau

Held-out masked-LM loss and accuracy over 5,000 steps



- First ~1,500 steps: loss stuck near the unigram entropy
- Only then does attention start to use context

ppl 77

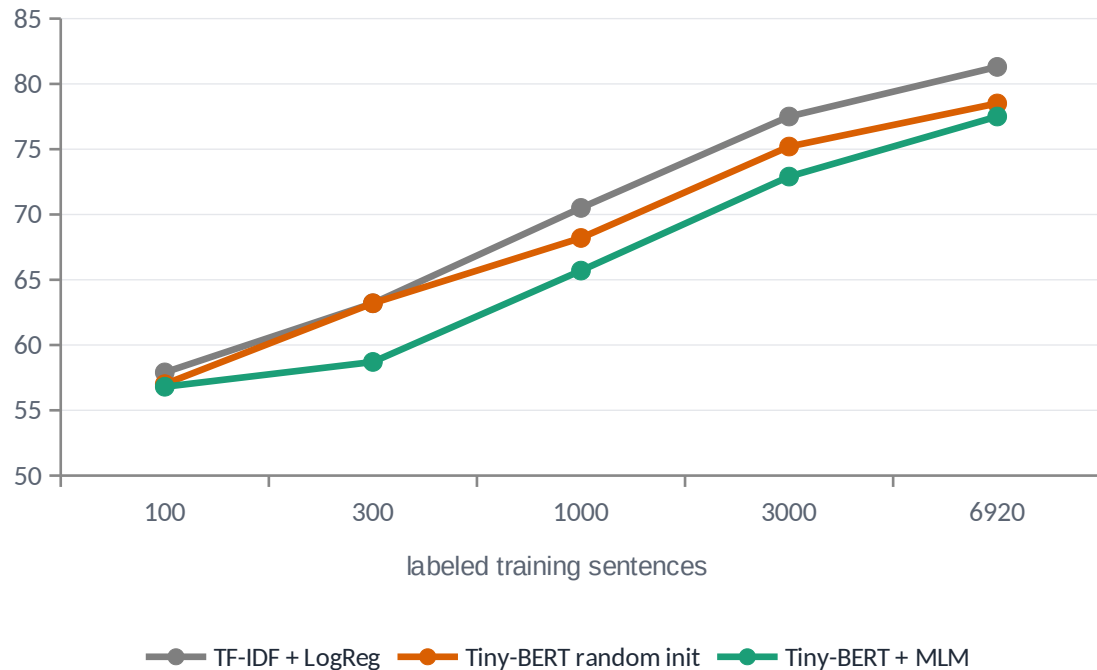
final held-out perplexity (from ~3,660)

29%

masked-token accuracy

Pre-training did not help; TF-IDF wins

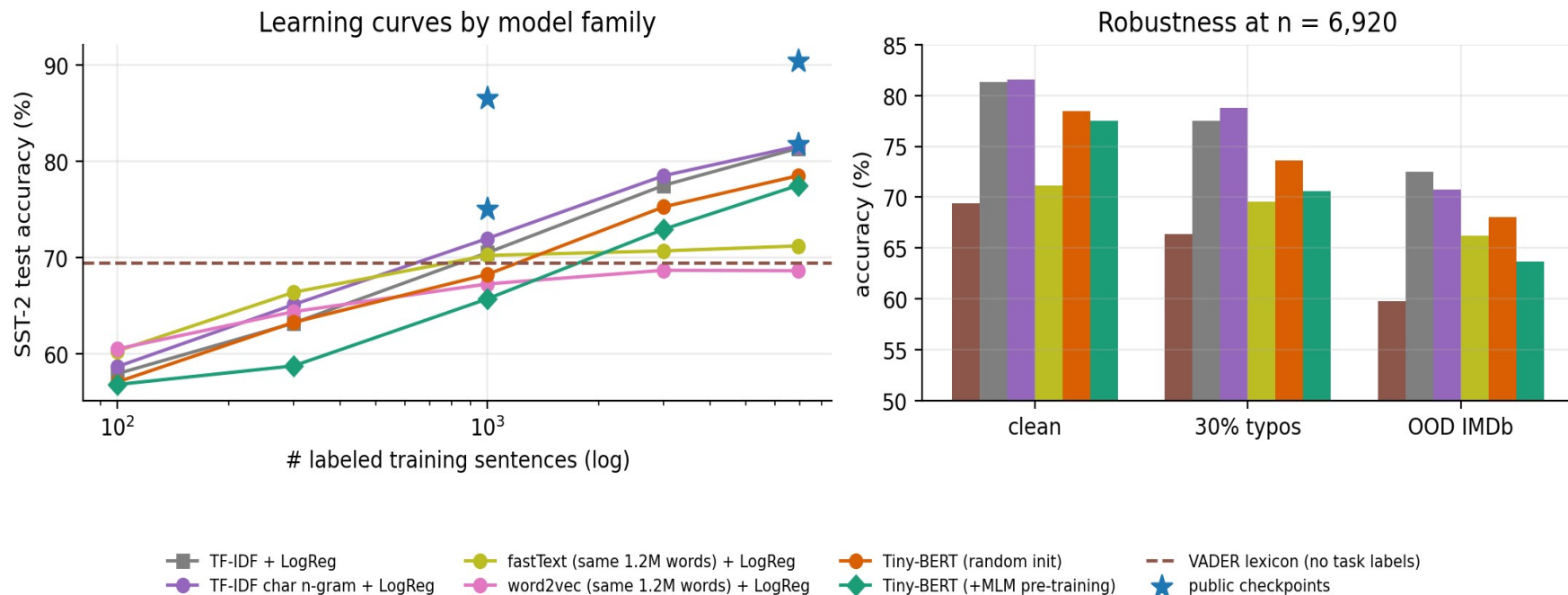
SST-2 test accuracy (%), mean of 3 seeds



- MLM vs random init: -0.3 to -4.5 pts at every budget (CIs exclude 0 at n=300,1k,3k)
- TF-IDF beats the pre-trained model by 3.9-4.8 pts from n=300 on

RQ6: the best bet depends on the label budget

SST-2 accuracy by model family (mean of 3 seeds)



- fastText (same 1.2M words) is the BEST model at 100-300 labels, beats MLM at n=1000 (70.2 vs 65.7) - then plateaus at 71.2%
- VADER (0 task labels): 69.4% - worth ~840 TF-IDF labels
- Character n-grams: best typo robustness (78.7% at 30% typos)

A currency for comparison: label-equivalence

How many TF-IDF labels is each model worth? (interpolated on the TF-IDF curve)

Model	n = 1,000	n = 6,920
VADER (0 labels)	~840	--
fastText (same words)	~960	~1,120
Tiny-BERT +MLM (ours)	~450	~3,000
BERT-Tiny (public)	~2,040	>6,920
DistilBERT (public)	>6,920	>6,920

0.45x

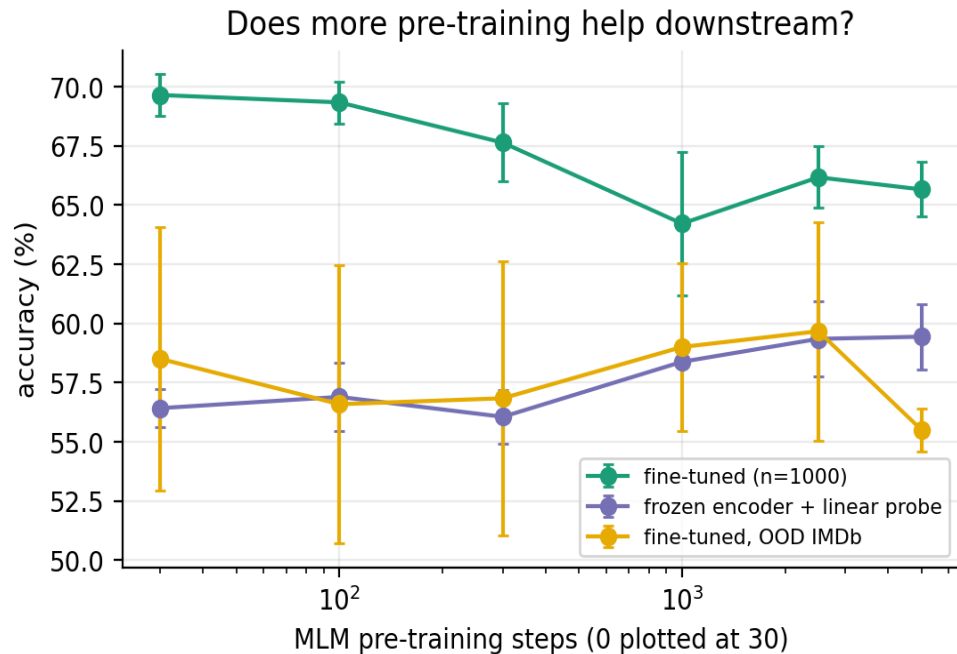
our pre-trained tiny BERT: 1,000 labels ~ 450 TF-IDF labels

6.9x

DistilBERT with 1,000 labels beats TF-IDF with all 6,920

RQ3: the probe improves, fine-tuning does not

Checkpoints after 0-5,000 MLM steps; n = 1,000 labels; 3 seeds



56.4 -> 59.4

frozen linear probe: MLM does store sentiment info

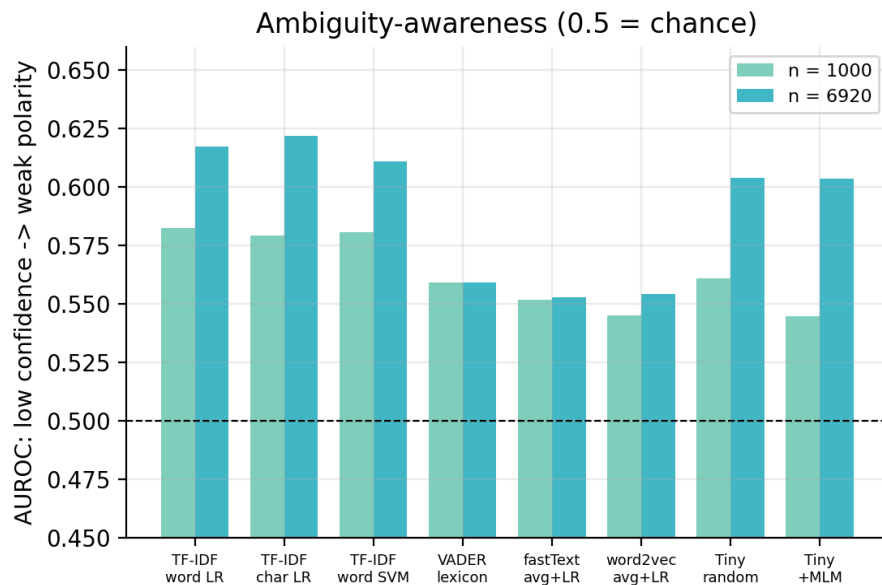
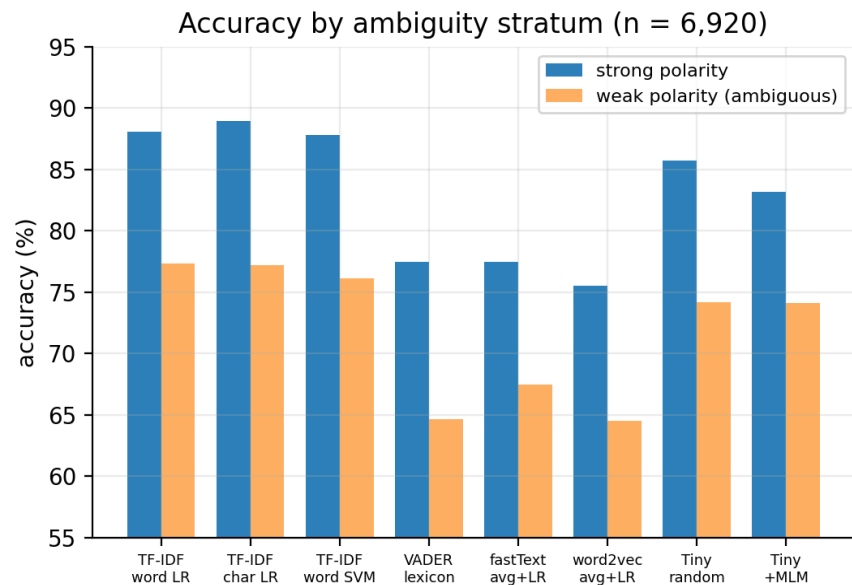
69.7 -> 65.7

fine-tuned accuracy falls with pre-training

A probe alone would have predicted a benefit.

RQ7: the deficit hides in plain sight

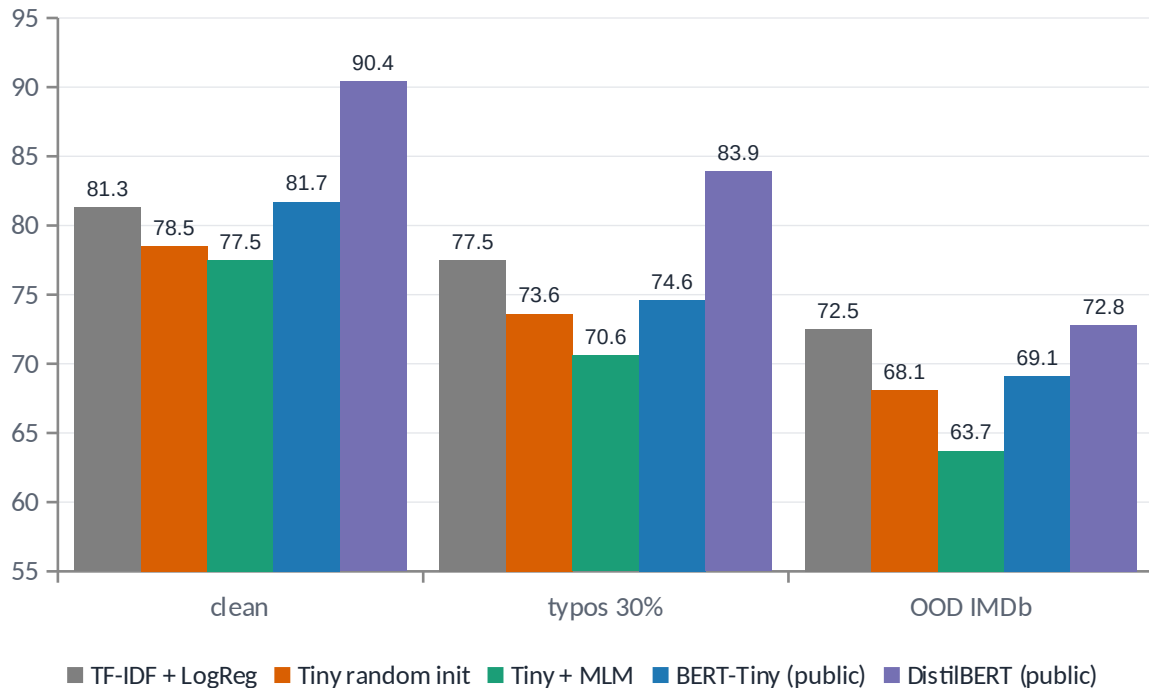
Where does pre-training lose, and does confidence track difficulty?



- -4.7 pts on STRONG (clear) sentences; only -1.3 on WEAK (ambiguous) ones - the deficit is concentrated on the easy cases
- Best ambiguity-awareness AUROC across ALL models: 0.62 (chance = 0.5) - nobody's confidence tracks ambiguity well
- Transformers on 1,000 labels are badly overconfident: ECE 23-26% vs 3-6% for TF-IDF

RQ2: typos and domain shift

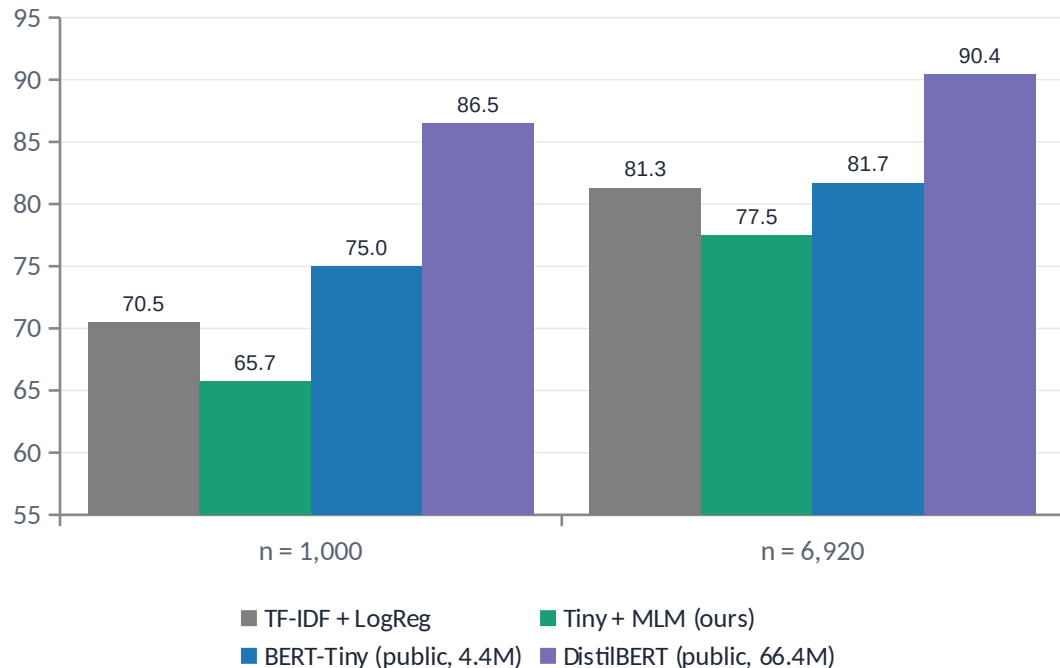
Accuracy (%) at n = 6,920 labeled sentences



- 30% typos: transformers lose 4.9-7.1 pts vs 3.8 for TF-IDF
- OOD: pre-training did not help ours; no public model beat TF-IDF

The twist: pre-training is a scale phenomenon

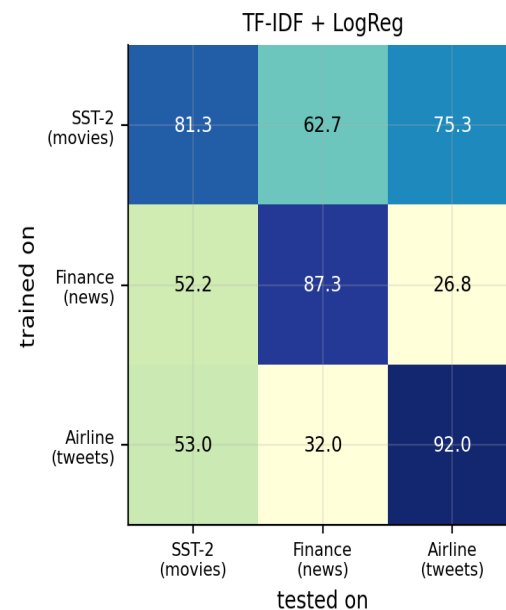
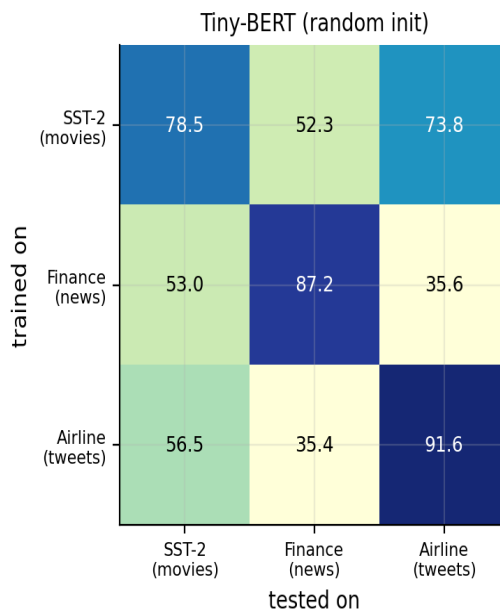
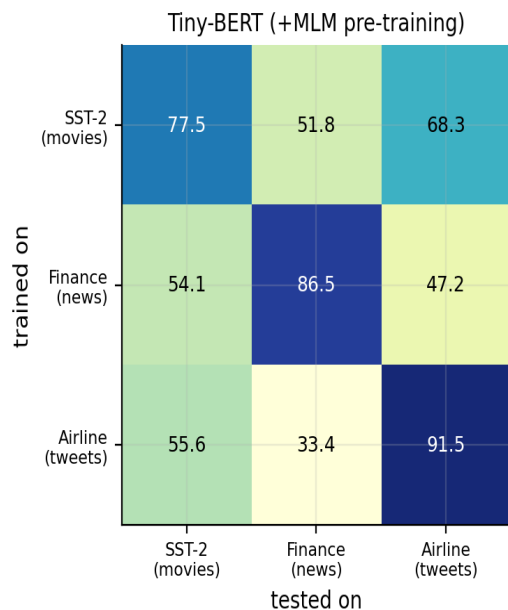
SST-2 accuracy (%), same fine-tuning code, 3 seeds



- BERT-Tiny has OUR depth and width: +9.3 pts at n=1k over our pre-trained model
- DistilBERT: +16/+9 pts over TF-IDF, ~26 min to fine-tune vs 37 s
- So: scale, not the recipe, is the missing ingredient

RQ8: does it travel? Cross-domain generalisation

Financial PhraseBank (finance news, real agreement) + Twitter Airline (tweets, real confidence)



- Small-budget result REPLICATES: fastText beats +MLM by +3.2 pts [+2.3,+4.1] at n=100 (airline); +MLM significantly WORSE than random init (-1.9 [-2.5,-1.2])
- Full-data result does NOT replicate: TF-IDF's SST-2 edge vanishes - all 3 families tie once a domain has enough labels
- Cross-domain transfer costs 15-60 pts everywhere; TF-IDF travels best (62.7% movies->finance); pre-training does not close the gap

Takeaways

Three findings, one field guide

It depends

fastText wins <1k labels; TF-IDF wins after; MLM never does

Where it fails

pre-training's damage sits on the EASY sentences

Don't trust it

no model's confidence tracks ambiguity (AUROC ≤ 0.62)

- Field guide: few hundred labels -> fastText; few thousand -> TF-IDF; public checkpoint available -> use it
- Scale, not the recipe, explains our negative result (BERT-Tiny, same architecture, does help)
- Next: more public checkpoints (code supports BERT-Mini/Small, MiniLM, TinyBERT, ELECTRA), tuned lr, second dataset

Backup: cost of each model

Single CPU thread; seed-0 full-data models

Model	Size (MB)	Latency (ms)	Sent./s	Acc. (%)
Tiny random init, FP32	4.71	1.03	4,914	78.3
Tiny +MLM, FP32	4.71	1.02	5,073	76.1
Tiny +MLM, INT8 dynamic	3.54	1.71	3,196	75.9
TF-IDF + LogReg	2.72	0.31	48,388	81.3
fastText + LogReg (est.)	~0.5	~0.5	~10,000	71.2

- INT8: -25% file size, accuracy unchanged within noise (single model)

- fastText row is an order-of-magnitude estimate, not separately measured

Backup: statistical evidence

Paired bootstrap, 95% CI over test sentences, seeds pooled; * = CI excludes 0

Comparison	n=1000	n=6920
+MLM - random init, STRONG sentences	-4.7* [-7.2,-2.2]	-2.6* [-4.8,-0.2]
+MLM - random init, WEAK sentences	-1.3 [-3.4,+0.8]	-0.1 [-2.2,+1.9]
+MLM - random init, TF-IDF LR (overall)	-2.5* [-4.2,-1.0]	-1.0 [-2.6,+0.5]

- Interaction (strong - weak) at n=1000: -3.5 [-6.7,-0.3], CI excludes 0
- Method: resample 1,821 test sentences 2,000 times; correctness difference averaged over seeds

Backup: limitations and threats to validity

What the results do and do not show

- Ambiguity proxy is SST-5 polarity STRENGTH, not annotator disagreement; 'weak' covers 62.8% of sentences
- Only 2 public checkpoints (BERT-Tiny, DistilBERT), untuned learning rates, predictions not stored (no paired tests for them)
- Embedding families untuned (fixed dim/epochs/pooling); no CNN/BiLSTM baseline
- 3 seeds; full 872-sentence dev set for early stopping even at n=100
- One task, one dataset, one language (SST-2, English)

Backup: reproducibility

Everything regenerates from logged runs

```
bash run_all.sh                # clean run of the whole study
python run_experiments.py --stage families # VADER, SVM, char n-gram,
word2vec/fastText
python ambiguity.py             # SST-5 strata, AUROC, ECE
python hf_benchmark.py --models ladder --tune # size ladder, tuned lr
```

- Resumable, stored predictions, paired statistics throughout
- Deliverables: code, technical docs, main report, 2-page report, ACM paper, this deck